

Lab06 – Containers and Kubernetes

17.04.2026

Basics:

- Introduction to containers, e.g. Docker: <https://docs.docker.com/get-started/>
 - Lightweight virtualization
 - Key concepts:
 - Dockerfile, Image, Container, Image repositories
- Kubernetes, container orchestration: <https://kubernetes.io/docs/concepts/overview/what-is-kubernetes/>
 - Cluster, Pods, Services, Jobs etc.
 - Used to manage a cluster of hosts/containers
 - Why do we need orchestration? What functionalities are provided by k8s, which exceed those of a simple dockerd daemon?
<https://kubernetes.io/docs/concepts/overview/what-is-kubernetes/>
 - Resource management
- Alternative implementations:
 - Containers: apptainer, podman, lxc
 - Container orchestration: OpenShift, Rancher, Nomad, commercial services: e.g. AWS ECS

Prerequisites

- Preferably a Linux environment (Linux vm is a good choice!)
- Docker
- Kubernetes cluster. While installing a full-fledged cluster is quite complicated, it's possible to prepare a development "mini k8s cluster" with help of minikube:
<https://minikube.sigs.k8s.io/docsstart/>
 - Yes, if you're following the default minikube installation guide you will be running k8s (a container orchestrator) inside of a container
 - Check your distro, it is possible that it already provides a minikube package!
- You can use other (if preferred) container engine or orchestrator

Tasks

1. The first container, the "greeter":
 - a. Create a greeter.sh script, which will print out a "hello world!" message to stdout, remember to add the executable bit: `chmod +x greeter.sh`
 - b. Create a Dockerfile for image based on your preferred Linux distro
 - i. It is best to use some other distro than the one used for the vm or host machine.
 - ii. Information on creating Dockerfiles/images can be found here:
https://docs.docker.com/get-started/02_our_app/

Please study the commands from the example dockerfile, note that this task requires only a subset of commands.

- iii. base images can be found here: <https://hub.docker.com/>
 - iv. include the "greeter.sh" script
 - c. Build an image based on your Dockerfile
 - d. Test/start the container in the interactive shell mode, verify if it's the chosen distro, this can be done by inspecting packages or a command like "lsb_release -a".
 - e. Start the container, supply the name "greeter.sh" script as an executable to run, observe the output. The container has its own filesystem, make sure you know the location where the script ended up.
2. Second container "web server":
- a. Create a Docker file for container based on a... container already including a web server (preferably https://hub.docker.com/_/httpd)
 - i. Provide a custom www page, include it in the container
 - b. Build and start the container, verify that the http port is accessible and responds with some content.

Note: Dockerfile is used as a recipe for building images, which in turn can be instantiated to become running containers. Images are stored in your local image repository. You can use sites like <https://hub.docker.com/> to share your images.

3. K8s cluster deployment:
- a. While containers are handy and easy to use, in most cases we'll be working on applications that are a bit more complex than ones created in pt. 1 and 2. It's often handy to have some redundancy, fail safe or scaling capabilities. Kubernetes can provide such functionalities. Here, we'll focus on running your apps managed by k8s.
 - b. Start the cluster,
 - c. Minikube, by default, uses its own internal Docker daemon. This daemon doesn't know anything about images build previously. Prepare your environment by directing it to access the internal docker daemon by using the "\$(minikube docker-env)" command and rebuild your images. This way images will be available within the k8s cluster.
(<https://minikube.sigs.k8s.io/docs/handbook/pushing/#1-pushing-directly-to-the-in-cluster-docker-daemon-docker-env>)
 - d. Create a deployment, in a form of a yaml file, for "web server", verify that the web server is running and responding to http requests. It might be necessary to explicitly declare the access port.
 - e. If you encounter issues where k8s always tries to download the image from registry (not using the version from a local repository), try setting the *imagePullPolicy* to "Never". The default value to *imagePullPolicy* changes depending on the version of image:
<https://kubernetes.io/docs/concepts/containers/images/#imagepullpolicy-defaulting>

Note: Did you know, that k8s can also execute "jobs", i.e. containers executing a specific task and exiting after that: <https://kubernetes.io/docs/concepts/workloads/controllers/job/>

Provide a report including the source code and usage examples with outputs (where applicable).